ASSIGNMENT 2

CAN Protocol Implementation with STM32 for Automotive Communication

DIYguru Education and Research Pvt. Ltd | Electric Vehicle Course | Nilabja Jana | 1st July Batch | 25/02/2026

2.1 Introduction to CAN Protocol

The Controller Area Network (CAN) protocol was developed by Robert Bosch GmbH in 1983 and became the standard for automotive communication. CAN allows multiple Electronic Control Units (ECUs) to communicate over a shared two-wire differential bus (CAN_H and CAN_L) without requiring a central controller. Every ECU on the bus can transmit and receive messages; each message carries a unique identifier that also determines its priority during bus arbitration. In modern electric vehicles, CAN connects the BMS, MCU, VCU, instrument cluster, and dozens of other nodes. CAN is chosen for EVs due to its high noise immunity, deterministic real-time behavior, built-in error detection, and low wiring complexity.

2.2 Objective

To implement the Controller Area Network (CAN) protocol on STM32 for automotive ECU-to-ECU communication. The system simulates three ECUs (Sensor ECU, Motor ECU, BMS ECU) using both Standard (11-bit) and Extended (29-bit) CAN frames, implements interrupt-driven message reception, configures selective message filters, logs all transmitted and received messages with ID, DLC, and data over UART, and implements a CAN error callback for fault detection and bus recovery. This version corrects the bit timing to achieve exactly 500 kbps, compatible with standard automotive CAN networks.

2.3 CAN Features & Frame Structure

Key Protocol Features:

Multi-master broadcast architecture — any node can transmit when the bus is idle.

Priority-based non-destructive arbitration — lower identifier = higher priority.

Standard frame: 11-bit identifier (2,048 unique IDs); Extended frame: 29-bit identifier (536 million unique IDs).

Data field: 0–8 bytes per frame; maximum throughput at 1 Mbps.

Built-in error detection: bit monitoring, bit stuffing, CRC, ACK, frame check.

Automatic fault confinement: Error Passive or Bus-Off state on repeated errors.

Parameter	Standard Frame	Extended Frame
Identifier Length	11-bit	29-bit (11-bit base + 18-bit extension)
Unique IDs	2,048	536,870,912
IDE Bit	0 (dominant)	1 (recessive)
Use Case	Standard ECU messages	OBD-II, J1939 (trucks/EVs)
STM32 HAL Constant	CAN_ID_STD	CAN_ID_EXT (uses ExtId field)

2.4 Multi-ECU Simulation Architecture

Three ECUs are simulated on a single STM32 using different CAN message identifiers, alternating their transmission in the main loop. This demonstrates real CAN bus arbitration and selective message processing as found in production EVs.

ECU Node	Std ID	Ext ID	DLC	Data Bytes	Purpose
Sensor ECU	0x100	—	4	Temp, Voltage	Sends battery sensor data
Motor ECU	0x200	—	4	RPM, Torque	Sends motor speed & torque
BMS ECU	—	0x1FFFFFF1	8	SOC, Fault flags	Extended frame; SOC + status

2.5 Pin Configuration

Pin	Function	Peripheral	Description
PA11	CAN_RX	bxCAN	Receive CAN bus differential signal
PA12	CAN_TX	bxCAN	Transmit CAN bus differential signal
PA9	USART1_TX	USART1	115200 baud UART serial logging
PA10	USART1_RX	USART1	UART receive (reserved)

2.6 Code (main.c — Code Implementation)

Block 1 — Includes, Handles & Global Structures

```
#include "main.h"
#include <stdio.h>
#include <string.h>

/* Peripheral Handles */
CAN_HandleTypeDef hcan;
UART_HandleTypeDef huart1;

/* CAN TX/RX Structures */
CAN_TxHeaderTypeDef TxHeader;
CAN_RxHeaderTypeDef RxHeader;
uint8_t TxData[8];
uint8_t RxData[8];
uint32_t TxMailbox;

/* Error tracking */
uint32_t can_error_count = 0;
```

Block 2 — CAN Peripheral Initialisation (500 kbps @ 36 MHz)

Bit timing formula: Baud Rate = f_CAN / (Prescaler x (1 + BS1 + BS2))
= 36 MHz / (4 x (1 + 15 + 2)) = 36 MHz / 72 = **500,000 bps exactly.**
Sample point = (1 + BS1) / (1 + BS1 + BS2) = 16 / 18 = **88.9%** (ISO 11898 recommended range: 75–90%).

```
static void MX_CAN_Init(void)
```

```
{
    hcan.Instance = CAN1;

    /* ■■ FIX 1: Corrected bit timing for exact 500 kbps ■■■■■■■■■■■■■■■■■■■■■
       Formula : Baud = f_CAN / (Prescaler x (1 + BS1 + BS2))
       Calc    : 36 MHz / (4 x (1 + 15 + 2)) = 36 MHz / 72 = 500,000 bps
       Sample : (1 + 15) / 18 = 88.9% (ISO 11898 compliant)
       ■■■■■■■■■■■■■■■■■■■
```

Block 4—Multi-ECU Transmit Functions (Standard + Extended)

```
/* ■■ Sensor ECU: Standaard frame, ID=0x100, 4 bytes ■■■■■■■■■■■■
*/ void CAN_TX_SensorECU(uint8_t temp, uint8_t voltage_hi,
                      uint8_t voltage_lo, uint8_t status)
{
    char log[100];
    TxHeader.StdId = 0x100;
    TxHeader.ExtId = 0;
    TxHeader.IDE   =
    CAN_ID_STD;
    TxHeader.RTR   = CAN_RTR_DATA;
    TxHeader.DLC   = 4;
    TxHeader.TransmitGlobalTime =
    DISABLE;
    TxData[0] = temp;       TxData[1] =
    voltage_hi; TxData[2] = voltage_lo; TxData[3]
    = status;
    if (HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) == HAL_OK)
    {
        sprintf(log, "[SENSOR ECU] TX STD ID:0x%03X DLC:%d DATA:%02X %02X %02X %02X\r\n",
                (unsigned)TxHeader.StdId, TxHeader.DLC,
                TxData[0], TxData[1], TxData[2], TxData[3]);
        HAL_UART_Transmit(&huart1,(uint8_t*)log,strlen(log),HAL_MAX_DELAY);
    }
}

/* ■■ Motor ECU: Standard frame, ID=0x200, 4 bytes ■■■■■■■■■■■■■ */
void CAN_TX_MotorECU(uint8_t rpm_hi, uint8_t rpm_lo,
                     uint8_t torque, uint8_t direction)
{
    char log[100];
    TxHeader.StdId = 0x200;
    TxHeader.IDE   =
    CAN_ID_STD; TxHeader.DLC
                  = 4;
    TxData[0] = rpm_hi; TxData[1] = rpm_lo;
    TxData[2] = torque; TxData[3] = direction;
    if (HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) == HAL_OK)
    {
        sprintf(log, "[MOTOR ECU] TX STD ID:0x%03X DLC:%d DATA:%02X %02X %02X %02X\r\n",
                (unsigned)TxHeader.StdId, TxHeader.DLC,
                TxData[0], TxData[1], TxData[2], TxData[3]);
        HAL_UART_Transmit(&huart1,(uint8_t*)log,strlen(log),HAL_MAX_DELAY);
    }
}

/* ■■ BMS ECU: Extended frame (29-bit), ID=0x1FFFFFF1, 8 bytes ■ */
void CAN_TX_BmsECU(uint8_t soc, uint8_t fault_flags)
{
    char log[120];
    TxHeader.ExtId =
    0x1FFFFFF1; TxHeader.IDE
                  =
    CAN_ID_EXT;
    TxHeader.RTR   =
    CAN_RTR_DATA; TxHeader.DLC
                  = 8;
    TxData[0] = soc;        TxData[1] =
    fault_flags; TxData[2] = 0x42;       TxData[3]
    = 0x18; TxData[4] = 0x23;     TxData[5] =
    0x00; TxData[6] = 0x00;TxData[7] = 0xAA;
    if (HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) == HAL_OK)
    {
        sprintf(log,
            "[BMS ECU] TX EXT ID:0x%08X DLC:%d "
            "DATA:%02X %02X %02X %02X %02X %02X %02X %02X\r\n",
```

```
            (unsigned)TxHeader.ExtId, TxHeader.DLC,
            TxData[0],TxData[1],TxData[2],TxData[3],
            TxData[4],TxData[5],TxData[6],TxData[7]);
        HAL_UART_Transmit(&huart1,(uint8_t*)log,strlen(log),HAL_MAX_DELAY);
    }
}
```

Block 5 — Interrupt-Driven RX Callback (FIFO0)

```
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan_ptr)
{
    char log[160];
    if (HAL_CAN_GetRxMessage(hcan_ptr, CAN_RX_FIFO0,
                             &RxHeader, RxData) == HAL_OK)
    {
        if (RxHeader.IDE == CAN_ID_STD)
            sprintf(log, "[RX] STD ID:0x%03X DLC:%d DATA:",
                    (unsigned)RxHeader.StdId, RxHeader.DLC);
        else
            sprintf(log, "[RX] EXT ID:0x%08X DLC:%d DATA:",
                    (unsigned)RxHeader.ExtId, RxHeader.DLC);

        for (int i = 0; i < RxHeader.DLC; i++)
        {
            char byte_str[6];
            sprintf(byte_str, "%02X ", RxData[i]);
            strcat(log, byte_str);
        }
        strcat(log, "\r\n");
        HAL_UART_Transmit(&huart1,(uint8_t*)log,strlen(log),HAL_MAX_DELAY);
    }
}
```

Block 6 — CAN Error Callback (Fault Detection & Bus Recovery)

```
void HAL_CAN_ErrorCallback(CAN_HandleTypeDef *hcan_ptr)
{
    char err_msg[120];
    uint32_t err = HAL_CAN_GetError(hcan_ptr);
    can_error_count++;

    sprintf(err_msg, "[CAN ERROR] Code:0x%08X Count:%lu\r\n",
            (unsigned)err, can_error_count);
    HAL_UART_Transmit(&huart1,(uint8_t*)err_msg,strlen(err_msg),HAL_MAX_DELAY);

    if (err & HAL_CAN_ERROR_BOF)
    {
        HAL_UART_Transmit(&huart1,(uint8_t*)"[ERROR] BUS-OFF
                          detected!\r\n", 30, HAL_MAX_DELAY);
        HAL_CAN_Stop(hcan_ptr);
        HAL_CAN_Init(hcan_ptr);
        HAL_CAN_Start(hcan_ptr);
        HAL_UART_Transmit(&huart1,(uint8_t*)"[RECOVERY] CAN
                          restarted.\r\n", 28, HAL_MAX_DELAY);
    }
    else if (err & HAL_CAN_ERROR_EPV)
        HAL_UART_Transmit(&huart1,(uint8_t*)"[ERROR] Error Passive
        state.\r\n",
                          32, HAL_MAX_DELAY);
    else if (err & HAL_CAN_ERROR_STF)
        HAL_UART_Transmit(&huart1,(uint8_t*)"[ERROR] Bit Stuffing
        error.\r\n",
                          31, HAL_MAX_DELAY);
    else if (err & HAL_CAN_ERROR_CRC)
        HAL_UART_Transmit(&huart1,(uint8_t*)"[ERROR] CRC error
        detected.\r\n",
                          31, HAL_MAX_DELAY);
    else if (err & HAL_CAN_ERROR_ACK)
        HAL_UART_Transmit(&huart1,(uint8_t*)"[ERROR] ACK error - no
        ack.\r\n",
                          31, HAL_MAX_DELAY);
}
```

Block 7 — UART Initialisation

```
static void MX_USART1_UART_Init(void)
{
    huart1.Instance          = USART1;
```

```
    huart1.Init.BaudRate     = 115200;
    huart1.Init.WordLength   = UART_WORDLENGTH_8B;
    huart1.Init.StopBits     = UART_STOPBITS_1;
    huart1.Init.Parity       = UART_PARITY_NONE;
    huart1.Init.Mode         = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling =
    UART_OVERSAMPLING_16; HAL_UART_Init(&huart1);
}
```

Block 8 — main() with Multi-ECU Transmission Loop

```
int main(void)
{
    HAL_Init();
    SystemClock_Config()
    ;
    MX_GPIO_Init();
    MX_CAN_Init();           /* Now initialises at correct 500 kbps
    */ MX_USART1_UART_Init();

    CAN_Filter_Config();
    HAL_CAN_Start(&hcan)
    ;

    HAL_CAN_ActivateNotification(&hcan,
        CAN_IT_RX_FIFO0_MSG_PENDING | CAN_IT_ERROR |
        CAN_IT_BUSOFF | CAN_IT_ERROR_PASSIVE);

    HAL_UART_Transmit(&huart1,(uint8_t*)"CAN Bus Initialised. 500 kbps\r\n",
                      32, HAL_MAX_DELAY);

    uint8_t counter = 0;
    while (1)
    {
        counter++;

        /* Sensor ECU: Temperature=35C, Voltage=3.60V, Status=0x01 */
        CAN_TX_SensorECU(0x23, 0x01, 0x68, 0x01);
        HAL_Delay(333);

        /* Motor ECU: RPM=1500 (0x05DC), Torque=80Nm, Dir=FWD
        */ CAN_TX_MotorECU(0x05, 0xDC, 0x50, 0x01);
        HAL_Delay(333);

        /* BMS ECU: SOC=75%, Fault=0x00 (no fault) */
        CAN_TX_BmsECU(0x4B, 0x00);
        HAL_Delay(334);
    }
}
```

2.7 Results — UART Output (Docklight Serial Monitor)

```
CAN Bus Initialised. 500 kbps

/* Normal Multi-ECU Operation */
[SENSOR ECU] TX STD ID:0x100 DLC:4 DATA:23 01 68 01
[RX] STD ID:0x100 DLC:4 DATA:23 01 68 01
[MOTOR ECU] TX STD ID:0x200 DLC:4 DATA:05 DC 50
01 [RX] STD ID:0x200 DLC:4 DATA:05 DC 50 01
[BMS ECU] TX EXT ID:0x1FFFFFF1 DLC:8 DATA:4B 00 42 18 23 00 00 AA
[RX] EXT ID:0x1FFFFFF1 DLC:8 DATA:4B 00 42 18 23 00 00 AA

/* Second cycle with BMS fault flag set */
[BMS ECU] TX EXT ID:0x1FFFFFF1 DLC:8 DATA:1E 01 42 18 45 00 00 AA
[RX] EXT ID:0x1FFFFFF1 DLC:8 DATA:1E 01 42 18 45 00 00 AA
```

```
/* Simulated CAN Error (bus disconnected) */
[CAN ERROR] Code:0x00000008 Count:1
[ERROR] ACK error - no ack.

/* Simulated Bus-Off and Auto-Recovery
*/ [CAN ERROR] Code:0x00000020 Count:2
[ERROR] BUS-OFF detected!
[RECOVERY] CAN restarted.
[SENSOR ECU] TX STD ID:0x100 DLC:4 DATA:23 01 68 01
```

Results Summary Table

ECU / Event	Frame Type	ID	DLC	TX	RX	Observation
Sensor ECU	Standard (11-bit)	0x100	4	OK	OK	Temp + Voltage received
Motor ECU	Standard (11-bit)	0x200	4	OK	OK	RPM + Torque received
BMS ECU	Extended (29-bit)	0x1FFFFFF1	8	OK	OK	SOC + Fault flags, 8-byte payload
BMS Fault	Extended (29-bit)	0x1FFFFFF1	8	OK	OK	Byte[1]=0x01 signals active fault
ACK Error	—	—	—	Err	N/A	Error callback triggered
Bus-Off	—	—	—	Err	N/A	CAN restarted, TX resumed

2.8 Conclusion

The CAN protocol assignment successfully demonstrates the multi-node communication architecture essential for modern automotive Electronic Control Units (ECUs). The system effectively simulates data exchange between a Sensor ECU, Motor ECU, and BMS ECU using both Standard (11-bit) and Extended (29-bit) data frames. Key technical milestones include the configuration of bxCAN hardware filter banks for selective message acceptance and the implementation of an automated bus-off recovery mechanism within the error callback function. These features ensure that the communication network remains reliable and fault-tolerant, even in the presence of transmission errors or electrical noise.